

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
C0 Assessed:	C05	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
C05 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	K. MRUDHULA	Reg.No.:	192524335
Department:	BTECH AI DS	Date:	31/08/26

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s

- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I K. MRUDHULA (192524335) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
K. MRUDHULA

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided
---------------	-----	--	--	-------------------------------	---------------------------

1. Construct an I/O Communication Mechanism for a Real-Time Embedded System

Given Constraints

The system must satisfy the following requirements:

- CPU utilization must be below 15%.
- Multiple sensors generate interrupts every 1 ms.
- Vectored interrupt handling must be used.
- Selection between memory-mapped I/O and I/O-mapped I/O must provide optimal latency.

Problem Understanding

A real-time embedded system receives data continuously from multiple sensors. Since the sensors generate interrupts every 1 ms, the processor must respond quickly without spending excessive CPU time handling I/O operations. The communication mechanism therefore needs an efficient interrupt architecture with low latency and limited CPU overhead.

Constraint Analysis

The most important constraint is the CPU utilization below 15%. Polling cannot be used because continuously checking sensors would consume considerable CPU time. Instead, interrupt-driven I/O should be used.

Since interrupts occur frequently, vectored interrupts are preferred because every interrupt source has a predefined interrupt vector. This allows the processor to directly identify the appropriate Interrupt Service Routine (ISR).

For I/O addressing, memory-mapped I/O is selected because device registers can be accessed using normal load/store instructions, providing a simple and low-latency interface.

Working

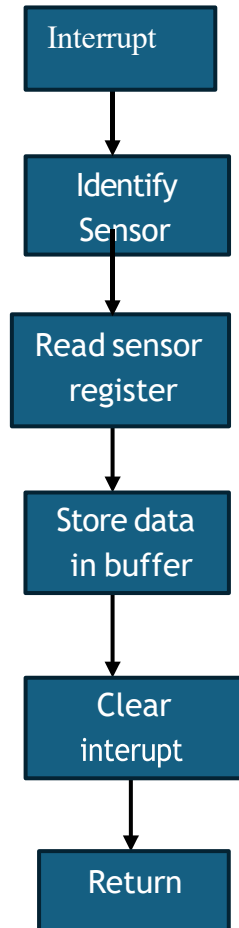
1. Each sensor is connected to the interrupt controller.
2. When a sensor requires service, it generates an interrupt.
3. The interrupt controller identifies the source.
4. A corresponding **interrupt vector** is supplied to the CPU.
5. The CPU saves the current execution state.
6. The CPU jumps directly to the appropriate ISR.
7. The ISR reads the sensor register using memory-mapped I/O.
8. The sensor data is stored in memory.
9. The interrupt flag is cleared.

10. The CPU returns to the interrupted program.

CPU Utilization

To keep CPU utilization below 15%, the ISR must be extremely short.

The ISR should perform only essential operations:



Any computationally intensive processing should be performed later as a background task instead of inside the ISR.

Why Memory-Mapped I/O?

Feature	Memory-Mapped I/O	I/O-Mapped I/O
Address space	Shared with memory	Separate I/O space
Instructions	Normal load/store	Special I/O instructions
Programming flexibility	High	Lower
Register access	Simple	Dedicated instructions

Memory-mapped I/O is selected because it provides direct register access using normal memory instructions and integrates efficiently with the CPU's addressing mechanism.

Justification

The proposed mechanism satisfies all constraints:

- CPU utilization can be maintained below 15% by using short ISRs.
- Frequent 1-ms sensor interrupts are handled using hardware interrupts.
- Vectored interrupts provide fast identification of interrupt sources.
- Memory-mapped I/O provides efficient register access.
- Sensor processing can be buffered and handled outside the ISR.

Result

Thus, a vectored interrupt-based, memory-mapped I/O mechanism provides a low-latency and CPU-efficient communication solution for the real-time embedded system.

2. Design an I/O Subsystem for High-Speed NVMe Storage

Given Constraints

- Storage throughput must be greater than 3 GB/s.
- Must use NVMe over PCIe.
- CPU should not handle bulk data transfer.
- DMA controller must be integrated with interrupt notification.

Problem Understanding

The system requires extremely high-speed storage communication. Traditional storage interfaces and CPU-controlled data transfers can introduce significant overhead. Therefore, NVMe over PCIe should be used along with DMA so that storage data can move directly between the NVMe device and main memory.

Working

The CPU does not directly copy storage data.

1. The CPU prepares an NVMe command.
2. The command is placed in an NVMe submission queue.
3. The CPU informs the NVMe controller about the new command.
4. The NVMe controller processes the request through PCIe.
5. DMA transfers data directly between the NVMe SSD and main memory.

6. The CPU remains free for other tasks.
7. After completion, the NVMe controller generates an interrupt.
8. The CPU handles the completion notification.
9. The completed data is then processed by the application.

4. Role of DMA

DMA is essential because the CPU should not handle bulk data transfer.

Without DMA:

NVMe → CPU → Memory

This requires CPU involvement for every transfer.

With DMA:

NVMe → DMA → Memory

The CPU only initializes and manages the operation.

5. Throughput Requirement

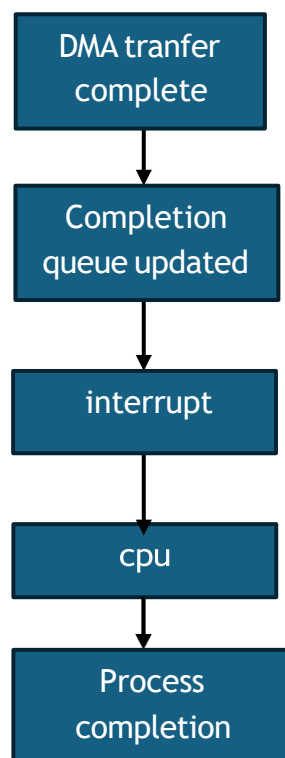
The required throughput is:

Throughput > 3 GB/s

PCIe-based NVMe provides the high-bandwidth communication path necessary to achieve multi-GB/s storage performance, provided that the selected PCIe generation, lane configuration, SSD and system implementation provide sufficient bandwidth.

6. Interrupt Notification

After completing a DMA transfer, the NVMe controller sends a completion notification/interrupt.



The interrupt mechanism prevents the CPU from continuously polling the storage device.

7. Justification

NVMe over PCIe is appropriate because it is designed for high-speed non-volatile storage and supports parallel command processing. DMA eliminates unnecessary CPU data copying, while interrupt notification informs the CPU only when an operation requires attention.

Result

Therefore, an NVMe-over-PCIe subsystem integrated with DMA and interrupt notification satisfies the >3 GB/s throughput requirement while minimizing CPU involvement.

3. Construct a Multi-Device Communication Architecture

Given Constraints

- 6 peripherals use USB.
- 2 high-speed devices use Thunderbolt.
- Interrupt starvation must be avoided.
- Both hardware and software interrupts must be used.
- Fair bus arbitration must be ensured.

Problem Understanding

The system contains eight external devices using two different high-speed interfaces. Because multiple devices may request CPU and bus resources simultaneously, the architecture must prevent one device from continuously occupying the bus or interrupting the processor.

USB Devices

The six USB peripherals are connected through a USB host controller:

USB Host Controller

- USB Device 1
- USB Device 2
- USB Device 3
- USB Device 4
- USB Device 5
- USB Device 6

The host controller schedules access to the USB devices.

Thunderbolt Devices

The two high-speed devices are connected through the Thunderbolt controller.

Thunderbolt Controller

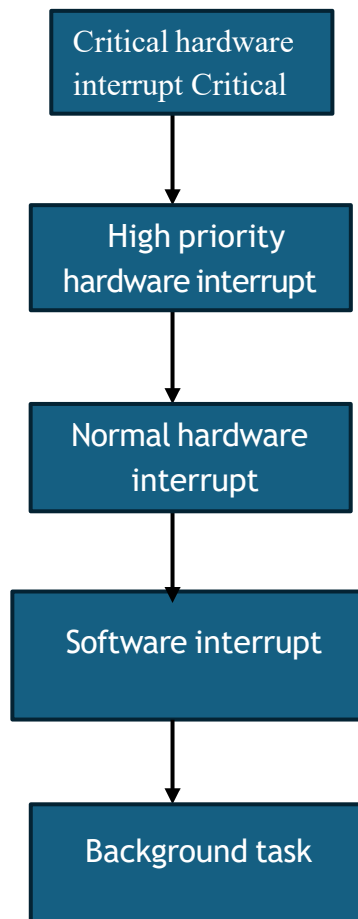
- High-Speed Device 1
- High-Speed Device 2

These devices receive suitable priority according to their latency and bandwidth requirements.

Avoiding Interrupt Starvation

Interrupt starvation occurs when low-priority devices continuously wait because high-priority interrupts are repeatedly serviced.

A suitable priority scheme is:



However, lower-priority requests should receive service within a bounded interval using priority aging or fair scheduling.

Hardware and Software Interrupts

Hardware interrupts are generated by physical devices such as USB and Thunderbolt peripherals.

Software interrupts are generated by programs or operating-system services for tasks that do not require immediate hardware response.

Example:

USB data ready → Hardware Interrupt

OS service request → Software Interrupt

Justification

The architecture:

- Supports all 6 USB devices.
- Supports 2 Thunderbolt devices.
- Uses hardware interrupts for device events.
- Uses software interrupts for software-generated requests.
- Prevents starvation through bounded priority scheduling.
- Uses fair bus arbitration.
- Allows high-speed devices to obtain appropriate bandwidth.

Result

Thus, a multi-controller architecture with prioritized interrupts and fair bus arbitration provides efficient and starvation-free communication for all eight devices.

4. Design a Data Acquisition System

Given Constraints

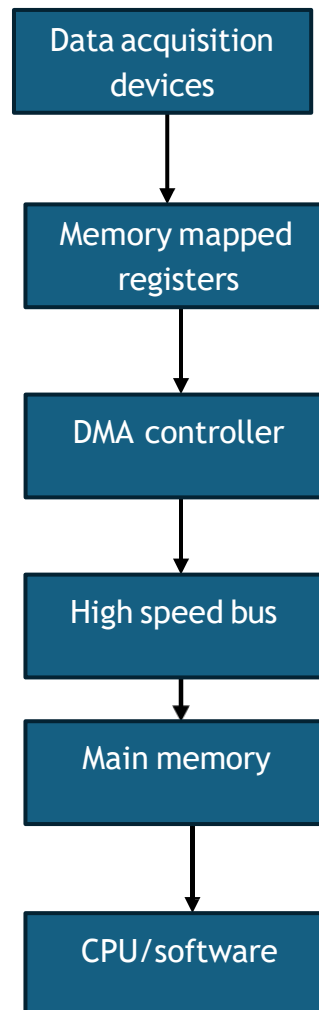
- Continuous data stream = 500 MB/s.
- Minimal CPU intervention.
- DMA transfer must be compared with interrupt-driven I/O.
- Device registers must use memory-mapped I/O.

Problem Understanding

The system receives a continuous data stream of 500 MB/s. Such a high data rate cannot be efficiently handled by the CPU using frequent interrupts because a large number of interrupts would consume processor resources.

Therefore, DMA is the preferred mechanism.

Proposed Architecture



Working

1. The acquisition device continuously produces data.
2. Its control and status registers are mapped into the CPU's memory address space.
3. The CPU configures the DMA controller.
4. DMA receives the data directly from the acquisition device.
5. DMA writes the data into a predefined memory buffer.
6. The CPU does not copy every data item.
7. After a block is transferred, DMA generates an interrupt.

8. The CPU processes the completed buffer while DMA can continue filling another buffer.

DMA vs Interrupt-Driven I/O

Parameter	DMA	Interrupt-Driven I/O
CPU involvement	Low	High
Suitable for 500 MB/s	Yes	No
Data transfer	Direct device-memory	CPU generally handles transfer
Interrupt frequency	Low, usually per block	High, potentially per event/data unit
CPU overhead	Low	High
Continuous high-speed stream	Excellent	Inefficient

Memory-Mapped I/O

The acquisition device registers are assigned memory addresses.

For example:

- 0x8000 – Control Register
- 0x8004 – Status Register
- 0x8008 – Data Register
- 0x800C – DMA Configuration

The CPU accesses these registers using normal memory read/write operations.

Justification

DMA is clearly better for the 500 MB/s continuous stream because transferring every data unit through the CPU would create excessive processor overhead. Interrupts can still be used, but only to notify the CPU after a block or buffer has been completed.

Result

A DMA-based data acquisition architecture with memory-mapped device registers and double buffering provides continuous 500 MB/s data transfer with minimal CPU intervention.

5. Construct an Interrupt Handling Mechanism

Given Constraints

- Must support nested interrupts.
- Maximum interrupt latency must be less than 10 μ s.
- Vectored interrupts must be used for high-priority devices.
- Software interrupts must be used for background tasks.

Problem Understanding

The system contains devices with different urgency levels. High-priority devices require rapid response, while background software tasks can tolerate longer delays. Therefore, the interrupt mechanism must support priority levels and nested interrupt processing.

Interrupt Priority

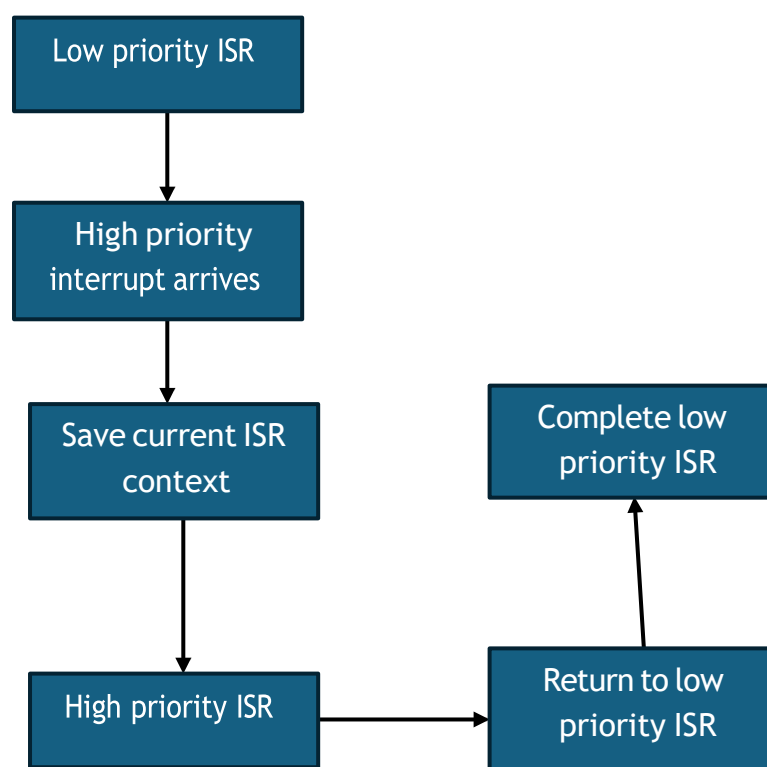
A priority structure can be defined as:

- Priority 1 $\rightarrow$ Emergency / critical device
- Priority 2 $\rightarrow$ High-speed I/O
- Priority 3 $\rightarrow$ Normal I/O
- Priority 4 $\rightarrow$ Background software tasks

A higher-priority interrupt is allowed to interrupt a lower-priority ISR.

Nested Interrupt Operation

Consider that the CPU is executing a low-priority ISR.



The CPU saves registers and processor status before entering the higher-priority ISR.

Vectored Interrupts

High-priority devices use vectored interrupts.

- Device 1 → Vector 01 → ISR 1
- Device 2 → Vector 02 → ISR 2
- Device 3 → Vector 03 → ISR 3

This avoids the need to scan multiple devices to determine the interrupt source and therefore reduces interrupt latency.

Software Interrupts

Software interrupts are assigned to background operations such as:

- Operating-system services
- Deferred processing
- Non-critical communication
- Background maintenance tasks

They should have lower priority than critical hardware interrupts.

Achieving <10 μ s Latency

To satisfy the maximum latency requirement:

1. Use vectored interrupts.
2. Use a dedicated interrupt controller.
3. Assign high priority to time-critical devices.
4. Keep ISRs short.
5. Avoid lengthy processing inside interrupt routines.
6. Use efficient context saving/restoring.
7. Allow higher-priority interrupts to preempt lower-priority ISRs.

Justification

Vectored interrupts reduce source-identification time, while priority-based nested interrupts ensure that critical devices are serviced immediately. Short ISRs reduce blocking time, and software interrupts keep background activities separate from urgent hardware events.

Result

Therefore, a priority-based nested vectored interrupt architecture can satisfy the required less-than-10- μ s maximum interrupt latency, while software interrupts efficiently handle background operations.